

Parameter estimation

See also: [Wentworth \(2025\) Chapter 10](#)

Modelling 2025 class notes

Sasha D. Hafner

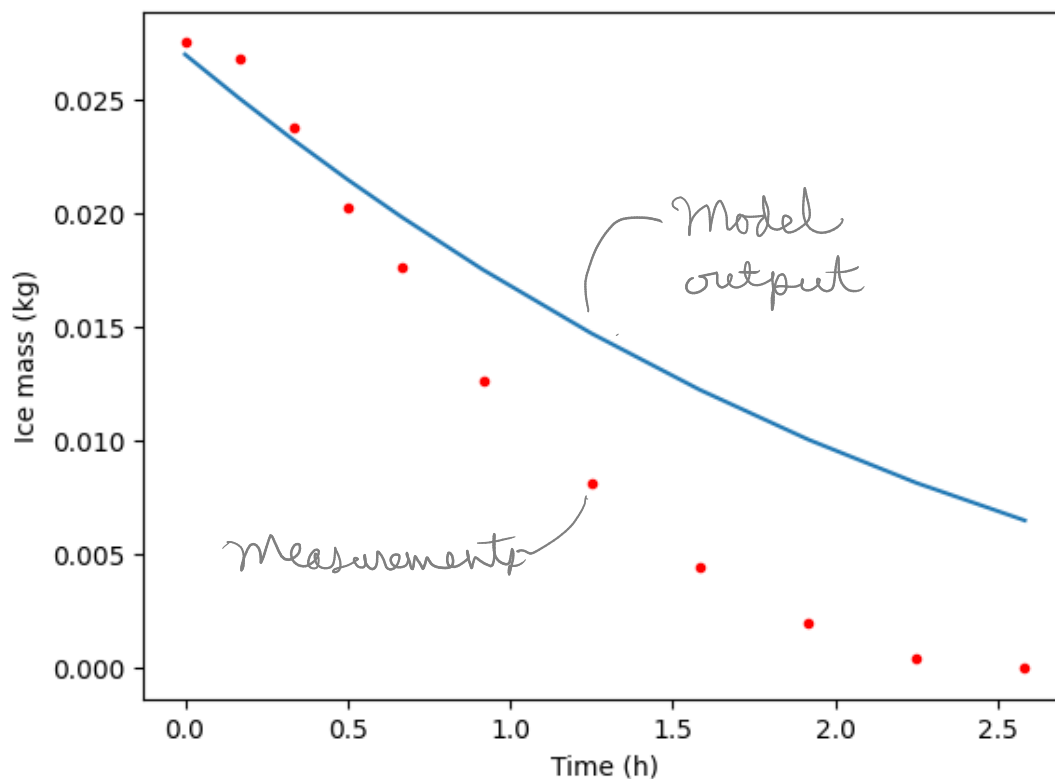
https://cdwentworth.github.io/CSBook/chapters/Ch10/Ch10_Least-squaresMethodsForData-fitting.html

Parameter estimation basics

Parameter estimation is about determining (estimating) the best (most accurate?) parameter values for a model based on comparing model output and measurements.

It is related to model validation.

Melting ice example:



The basic idea is to identify the parameter values that provide the best fit to the measurements.

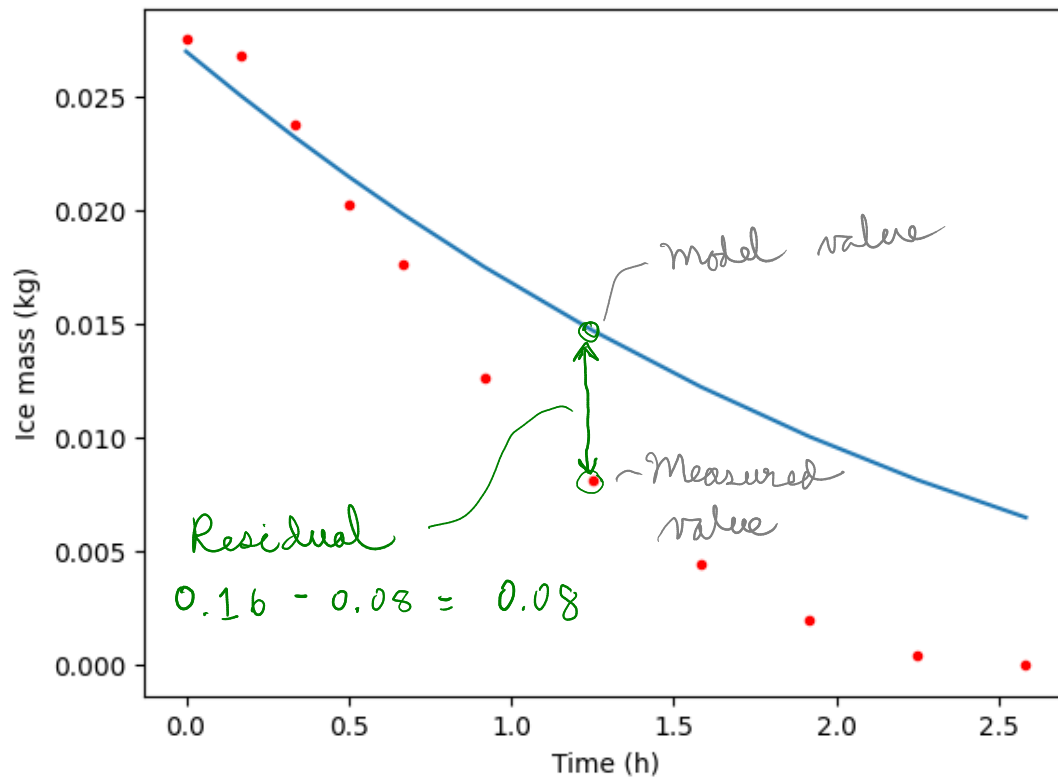
There are different ways to define "best fit", but we will focus on one common approach.

Warning: terms vary a lot! Model output may be called "predictions", "model(led) values", "theoretical values". Measurements may be called "observations" or just "data" (which is vague).

Model verification and validation should be done before parameter estimation! Otherwise results (parameter values) may be nonsense.

Residuals

Residuals quantify the difference between measurements and the model.



We'll use: $\text{residual} = \text{model value} - \text{measured value}$.

These are the same residuals that are used in the model fit statistics NS ME, MBE, MAE, RMSE.

Note that residuals are also called "model error" and are defined as measured - model in some field (notably all of statistical modeling!).

Loss function or objective function

Let's look at the melting ice model function signature.

```
def melt(mass_0,  
        temp_air,  
        h,  
        t_range,  
        t_step,  
        dens = 920,  
        temp_freeze = 0,  
        Hf = 333000  
):
```

One parameter: the convection heat transfer coefficient

So that is the parameter we will estimate.

"Manual approach": We could just try different values and see what we get. We did some of this with the demo for class 18:

- * Ice melting model validation demonstration (class 18) part 1: <https://youtu.be/UX1QXEocOc0>
- * Ice melting model validation demonstration (class 18) part 2: <https://youtu.be/Ruu7LvyY-V4>

But we cannot be sure we have the "best" value, and that approach is generally less efficient than using an algorithm.

An algorithm needs some quantitative measure of model fit. Common choices are:

- * Sum of squares = SSR = sum of squared residuals
- * sum of absolute deviation

These are examples of a

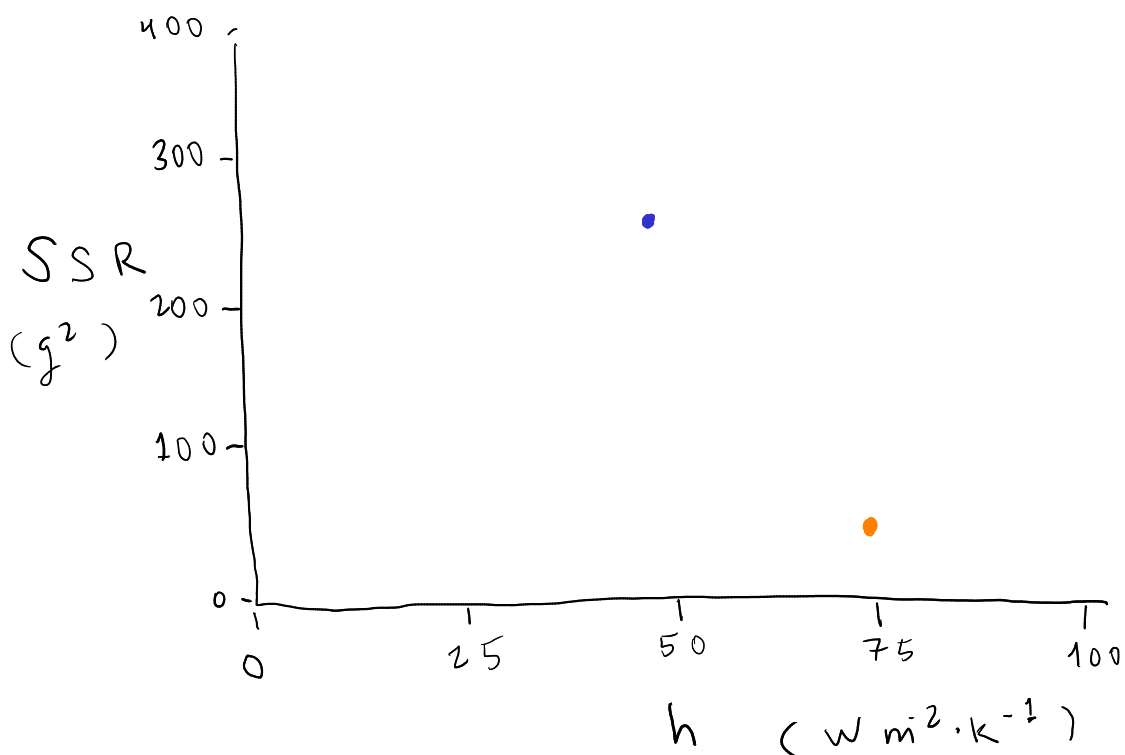
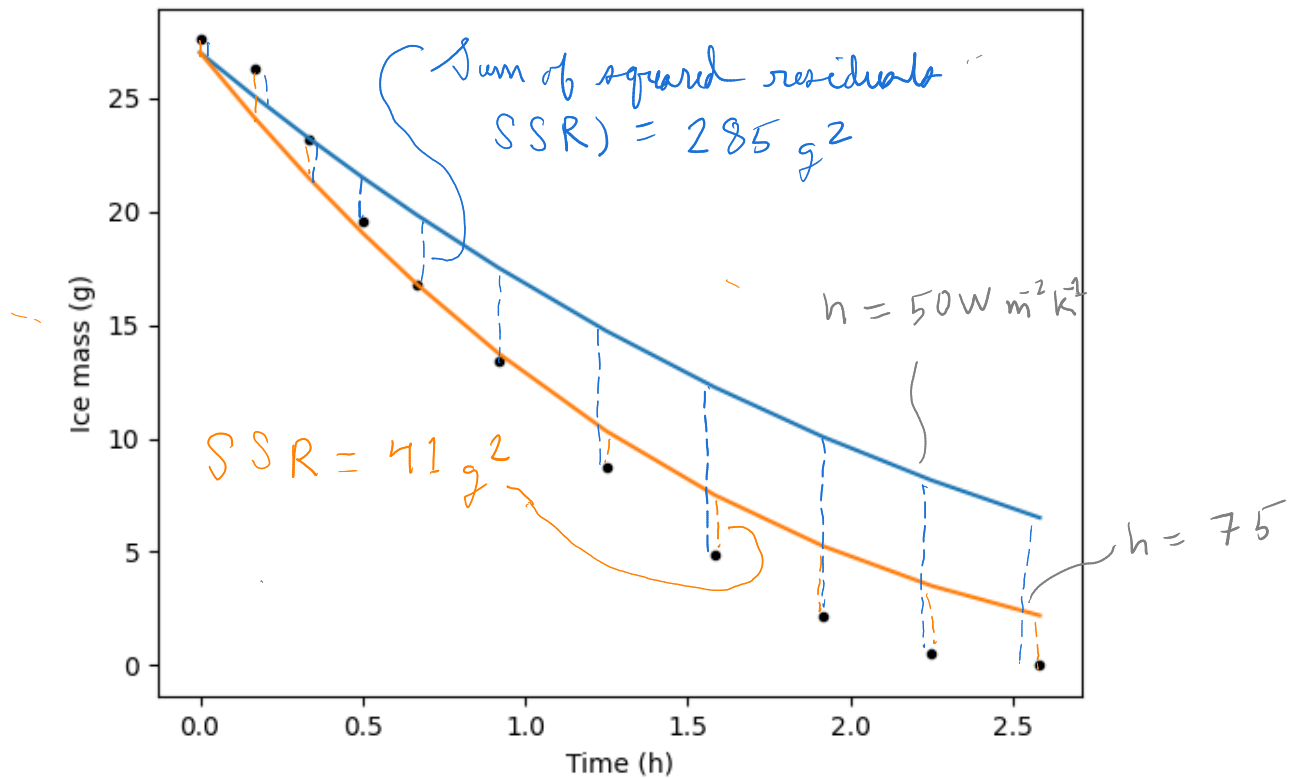
cost function or objective function: some function that summarizes model fit as a single numeric value (scalar), which is minimized in a parameter estimation process

Conceptual optimization

Parameter estimation is an application of optimization, which works by repeated evaluation of an objective function, updating the "guess" each time based on how the values change.

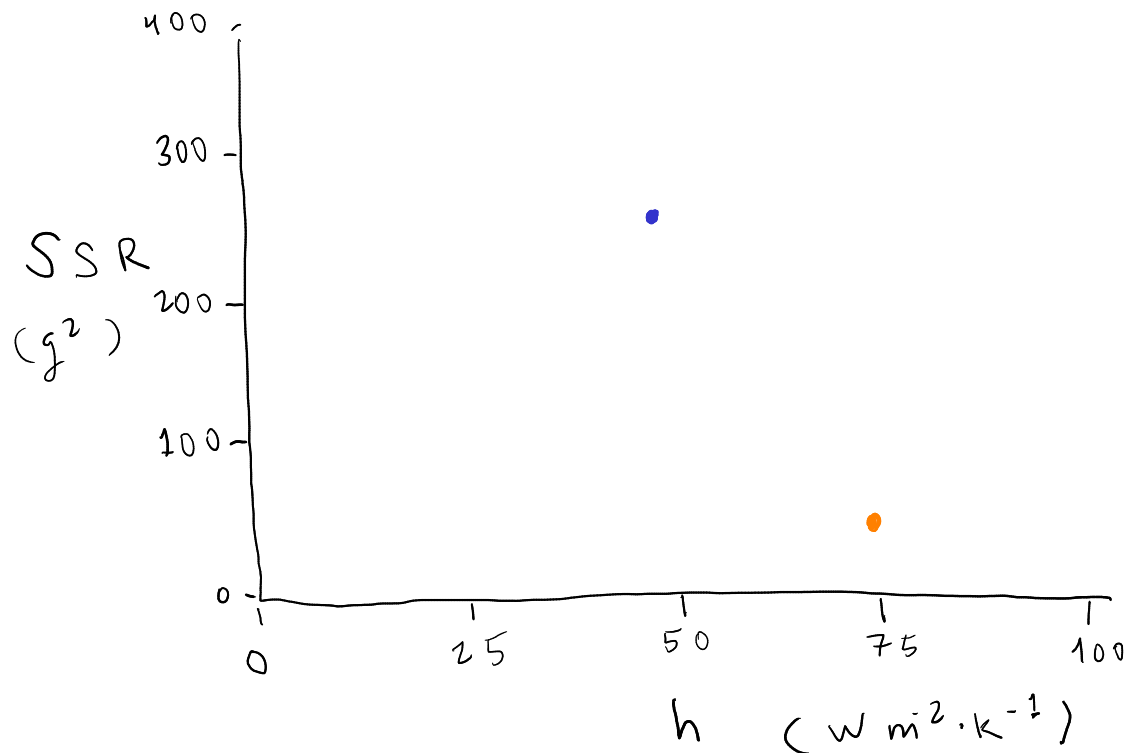
Cost function

```
>>> sum((meas.ice_g_1 - 1000 * pred01["m"])**2)
285.47779472097403
>>> sum((meas.ice_g_1 - 1000 * pred02["m"])**2)
40.79035590444834
```

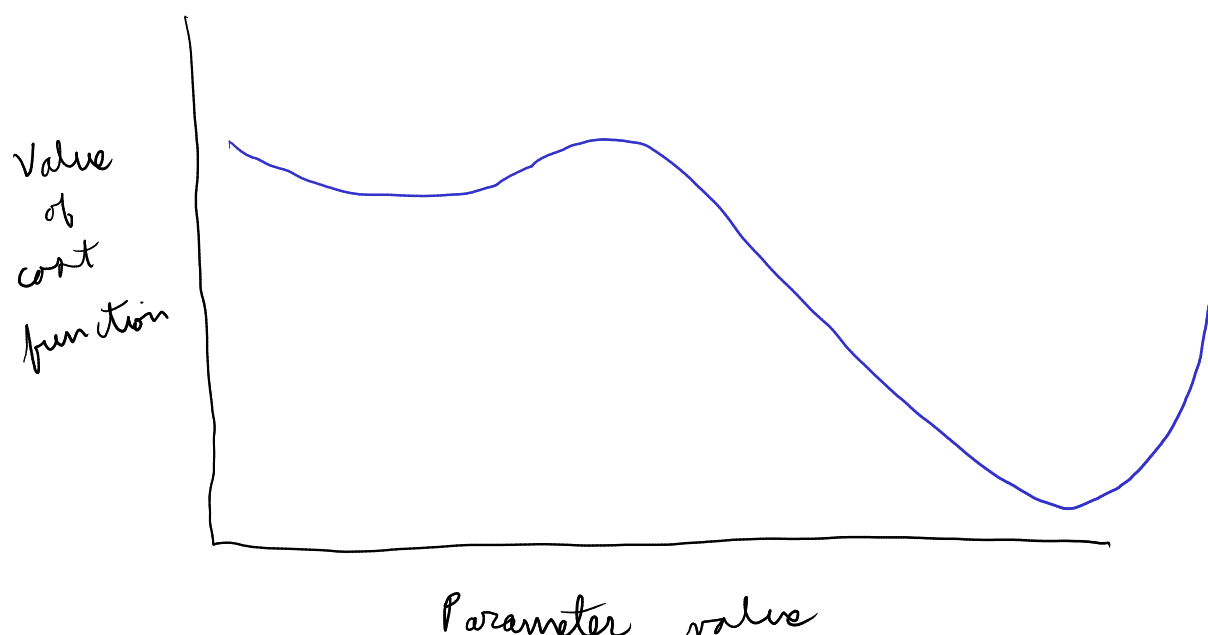


There are a few different ways to figure out the next guess, but the basic idea is for the optimization algorithm to figure out the shape of the curve, and try to find where it reaches a minimum

Each point is calculated from **all** residuals



Ideally we would like a global minimum, i.e., the best possible parameter value. But a local minimum is always a risk.



What to do?

- * Use a good starting guess
- * Check that result does not depend on starting guess (try more than one)

Note that the terms "objective function", "cost function", and "loss function" are sometimes used interchangeably.

Our usage:

- * Objective function: the function to be optimized (could even be maximized)
- * Cost function: a function to be minimized
- * Loss function: a function that adjusts the individual residuals (follow `least_squares()` usage)